

Classes livre et compte

SOMMAIRE

La Classe Livre.....	3
Introduction.....	3
Exercice 1 :	3
Déclaration des Attributs.....	3
Exercice 2 :	4
Le Constructeur.....	4
Exercice 3 :	5
Implémentation des Méthodes Getters & Setters.....	5
Exercice 4 :	6
Exercice 5 :	7
Résultat	9
La Classe Compte.....	10
1. Définir une classe Client.....	10
3. Définir un constructeur.....	11
4. Définir un constructeur.....	11
5. Définir la méthode Afficher.....	12
6. Créer une classe Compte.....	13
7. Définir à l'aide des propriétés les méthodes d'accès.....	14
8. Définir un constructeur permettant de créer un compte.....	15
9. Ajouter à la classe Compte les méthodes suivantes :.....	15
10. Créer un programme de test pour la classe Compte.....	18
10. Résultat final.....	19

La Classe Livre

Exercice 1 : Définir une classe Livre avec les attributs suivants : Titre, Auteur (Nom complet), Prix.

Déclaration des Attributs

Nous avons déclaré les trois attributs requis comme étant privés (**private**). Cela assure ce que l'on appelle "l'encapsulation", protégeant ainsi les données d'une modification externe. ↴

```
5 // Attributs privés
6
7 private String titre;
8 private String auteur;
9 private double prix;
10
```

Exercice 2 : Définir un constructeur permettant d'initialiser les attributs de la méthode par des valeurs saisies par l'utilisateur.

Le Constructeur

Nous avons défini un constructeur. Le rôle du constructeur est d'initialiser l'objet Livre dès sa création avec des valeurs spécifiques pour le **titre**, l'**auteur** et le **prix**. Le constructeur est la méthode spéciale utilisée pour créer et initialiser les objets

Il prend en paramètres les trois attributs. L'utilisation de **this** permet de distinguer les paramètres locaux des attributs de l'objet. ↴

```
// Attributs privés
private String titre;
private String auteur;
private double prix;

// Constructeur avec tous les attributs
public Livre(String titre, String auteur, double prix) {
    this.titre = titre;
    this.auteur = auteur;
    this.prix = prix;
}
```



Exercice 3 : Définir à l'aide des propriétés les méthodes d'accès aux différents attributs de la classe.

Implémentation des Méthodes d'Accès (Getters & Setters)

Pour lire ou modifier les attributs privés, nous avons défini les méthodes d'accès.

- **Getters** : Les méthodes **get**, permettent de lire la valeur d'un attribut de l'extérieur.
- **Setters** : Les méthodes **set**, permettent de modifier la valeur d'un attribut de l'extérieur. ↴

```
// Getters
public String getTitre() {
    return titre;
}

public String getAuteur() {
    return auteur;
}

public double getPrix() {
    return prix;
}

// Setters

public void setTitre(String titre) {
    this.titre = titre;
}

public void setAuteur(String auteur) {
    this.auteur = auteur;
}

public void setPrix(double prix) {
    this.prix = prix;
}
```

Exercice 4 : Définir la méthode Afficher () permettant d'afficher les informations du livre en cours.

Nous avons défini la méthode **afficher** qui génère une ligne de sortie formatée, affichant les trois attributs de l'objet courant. ↴

```
public String getAuteur() {
    return auteur;
}

public double getPrix() {
    return prix;
}

// Setters

public void setTitre(String titre) {
    this.titre = titre;
}

public void setAuteur(String auteur) {
    this.auteur = auteur;
}

public void setPrix(double prix) {
    this.prix = prix;
}

// Méthode d'affichage
public void afficher() {
    System.out.println("Titre: " + titre + ", Auteur: " + auteur + ", Prix: " + prix);
}
```



Exercice 5 : Écrire un programme testant la classe Livre.

Saisie et création du livre 1

Cette étape consiste à écrire le programme principal pour mettre en action notre classe **Livre**. Elle se déroule dans un fichier qui nous avons nommé **TestLivre**.

Nous demandons les trois informations (Titre, Auteur, Prix) et les stockons dans des variables temporaires. Ensuite, nous créons le premier objet **Livre** en utilisant le constructeur que nous avons défini à l'Étape 1. ↴

```
System.out.println("Livre 1:");
System.out.print("Saisir le titre: ");
String titre1 = input.nextLine();

System.out.print("Saisir l'auteur: ");
String auteur1 = input.nextLine();

System.out.print("Saisir le prix: ");
double prix1 = input.nextDouble();
input.nextLine();

Livre livre1 = new Livre(titre1, auteur1, prix1);
```

Affichage du livre 1

Nous devons prouver que l'objet a bien stocké les données. Pour respecter la séquence de l'exemple d'exécution, nous combinons deux méthodes d'affichage :

1. Affichage Détaillé : Nous utilisons les **get** pour *extraire* la donnée et l'afficher avec une phrase explicite. Cela prouve que l'encapsulation fonctionne et que les données sont accessibles.
2. Affichage Résumé : Nous appelons la méthode que l'objet contient lui-même. ↴

```
System.out.print("Saisir l'auteur: ");
String auteur1 = input.nextLine();

System.out.print("Saisir le prix: ");
double prix1 = input.nextDouble();
input.nextLine();

Livre livre1 = new Livre(titre1, auteur1, prix1);

//Affichage
System.out.println("Le titre est " + livre1.getTitre());
System.out.println("L'auteur est " + livre1.getAuteur());
System.out.println("Le prix est " + livre1.getPrix());
```



Saisie et création du livre 2

Nous répétons les mêmes étapes pour le livre 2. ↴

```
// livre 2

System.out.println("\nLivre 2:");
System.out.print("Saisir le titre: ");
String titre2 = input.nextLine();

System.out.print("Saisir l'auteur: ");
String auteur2 = input.nextLine();

System.out.print("Saisir le prix: ");
double prix2 = input.nextDouble();
input.nextLine();

Livre livre2 = new Livre(titre2, auteur2, prix2);
```

```
//Affichage

System.out.println("Le titre est " + livre2.getTitre());
System.out.println("L'auteur est " + livre2.getAuteur());
System.out.println("Le prix est " + livre2.getPrix());
```

Résultat ↴

```
Livre 1:
Saisir le titre: Germinal
Saisir l'auteur: Emile Zola
Saisir le prix: 4
Le titre est Germinal
L'auteur est Emile Zola
Le prix est 4.0
Titre: Germinal, Auteur: Emile Zola, Prix: 4.0
```

La Classe Compte

Pour cet exercice, l'objectif était de mettre en pratique la POO en modélisant un système bancaire simple.

1. Définir une classe Client avec les attributs suivants: CIN, Nom, Prénom, Tél.

Nous avons déclaré la classe **Client** et défini les quatre variables privées (**private**) toujours dans le but de protéger les données. ↴

```
package classecompte;

public class Client {

    // Attributs

    private String CIN;
    private String Nom;
    private String Prenom;
    private String Tel;
```

3. Définir un constructeur permettant d'initialiser tous les attributs

Nous avons créé deux constructeurs

Le premier initialise tous les attributs.

Le deuxième initialise seulement le **CIN**, **Nom** et **Prénom**. ↴

```
public class Client {  
  
    // Attributs  
    private String CIN;  
    private String Nom;  
    private String Prenom;  
    private String Tel;  
  
    // Constructeur avec tous les attributs  
    public Client(String CIN, String Nom, String Prenom, String Tel) {  
        this.CIN = CIN;  
        this.Nom = Nom;  
        this.Prenom = Prenom;  
        this.Tel = Tel;  
    }  
}
```



4. Définir un constructeur permettant d'initialiser le CIN, le nom et le prénom.

```
private String CIN;  
private String Nom;  
private String Prenom;  
private String Tel;  
  
// Constructeur avec tous les attributs  
public Client(String CIN, String Nom, String Prenom, String Tel) {  
    this.CIN = CIN;  
    this.Nom = Nom;  
    this.Prenom = Prenom;  
    this.Tel = Tel;  
}  
  
// Constructeur avec CIN, Nom, Prenom  
public Client(String CIN, String Nom, String Prenom) {  
    this.CIN = CIN;  
    this.Nom = Nom;  
    this.Prenom = Prenom;  
    this.Tel = "";  
}  
}
```



Pour permettre la lecture et la modification des attributs, nous avons défini les **Getter** et les **Setters**. ↴

```
// Getters et Setters

public String getCIN() {
    return CIN;
}

public void setCIN(String CIN) {
    this.CIN = CIN;
}

public String getNom() {
    return Nom;
}

public void setNom(String Nom) {
    this.Nom = Nom;
}

public String getPrenom() {
    return Prenom;
}

public void setPrenom(String Prenom) {
    this.Prenom = Prenom;
}

public String getTel() {
    return Tel;
}

public void setTel(String Tel) {
    this.Tel = Tel;
}
```

5. Définir la méthode Afficher () permettant d'afficher les informations du Client en cours.

La méthode **Afficher()** est implémentée pour les informations du client. Elle sera appelée par la classe **Compte** pour afficher les détails du propriétaire. ↴

```
public void setPrenom(String Prenom) {
    this.Prenom = Prenom;
}

public String getTel() {
    return Tel;
}

public void setTel(String Tel) {
    this.Tel = Tel;
}

// 5. Affichage

public void Afficher() {
    System.out.println("CIN: " + this.CIN);
    System.out.println("NOM: " + this.Nom);
    System.out.println("Prénom: " + this.Prenom);
    System.out.println("Tél: " + this.Tel);
}
```



6. Créer une classe Compte caractérisée par son solde et un code qui est incrémenté lors de sa création ainsi que son propriétaire qui représente un client.

Nous avons défini les attributs principaux (**solde**, **code**, **proprietaire**). L'attribut **proprietaire** est un objet de type **Client**, créant le lien entre les deux classes.

Pour assurer que le numéro de compte est unique et s'incrémente lors de chaque création, nous avons utilisé un compteur statique (**nbComptes**).

```
1 package classecompte;
2
3 public class Compte {
4
5     // Attributs
6
7     private double solde;
8     private int code;
9     private Client proprietaire;
```

```
    // Attribut statique
    private static int nbComptes = 0;
```

7. Définir à l'aide des propriétés les méthodes d'accès aux différents attributs de la classe.

Nous allons définir les **Getters** en respectant la consigne qui dit que le numéro de compte et le solde sont en lecture seule. Nous créons donc un Getter pour chacun, sans Setter.

```
// Getters / Setters

// Lecture seule

public int getCode() {
    return code;
}

public double getSolde() {
    return solde;
}

// Getterpropriétaire

public Client getProprietaire() {
    return proprietaire;
}
```

8. Définir un constructeur permettant de créer un compte en indiquant son propriétaire.

A présent nous devons créer le constructeur qui prend le "Client" en paramètre. À la création de l'objet, ce constructeur va :

- Augmenter la variable statique `nbComptes`.
- Attribuer cette nouvelle valeur au `code` du compte.
- Initialiser le `solde` à 0.

```
public Compte(Client proprietaire) {  
    nbComptes++;  
    this.code = nbComptes;  
    this.solde = 0;  
    this.proprietaire = proprietaire;  
}
```

9. Ajouter à la classe Compte les méthodes suivantes :

Une méthode permettant de **Créditer()** le compte, prenant une somme en paramètre.

Créons la méthode `Crediter`. Elle prendra un seul paramètre (`somme`) et aura pour unique fonction d'ajouter cette somme au `solde` actuel du compte. C'est l'équivalent d'un dépôt d'argent.

```
// Méthodes  
  
public void Crediter(double somme) {  
    this.solde += somme;  
}
```

Une méthode permettant de Créditer() le compte, prenant une somme et un compte en paramètres. créditant le compte et débitant le compte passé en paramètres.

Pour ajouter l'argent à notre compte, nous allons appeler la première méthode **Crediter(somme)**

Nous appelons la méthode **Debiter(somme)** sur le **"autreCompte"** passé en paramètre pour retirer l'argent.

```
public void Crediter(double somme, Compte autreCompte) {  
    this.Crediter(somme);  
    autreCompte.Debiter(somme);  
}
```

Une méthode permettant de Debiter() le compte, prenant une somme en paramètre.

La consigne nous demande de créer la méthode **debiter()**. Elle prend un seul paramètre et a pour unique fonction de soustraire cette somme du **solde** actuel du compte. C'est l'équivalent d'un retrait d'argent.

```
public void Debiter(double somme) {  
    this.solde -= somme;  
}
```


Une méthode permettant de Débiter() le compte, prenant une somme et un compte bancaire en paramètres, débitant le compte et créditant le compte passé en paramètres.

Nous allons appeler la première méthode **Debiter(somme)** pour retirer l'argent de notre compte (**this**).

Nous allons appeler la méthode **Crediter(somme)** sur le "**autreCompte**" passé en paramètre pour ajouter l'argent au compte destinataire.

```
public void Debiter(double somme, Compte autreCompte) {  
    this.Debiter(somme);  
    autreCompte.Crediter(somme);  
}
```

Une méthode qui permet d'afficher le résumé d'un compte.

Afin de regrouper toutes les informations essentielles nous allons créer la méthode correspondante. Elle affichera le numéro et le solde du compte, puis elle appellera la méthode **Afficher()** de notre objet **proprietaire** pour obtenir les détails du client.

```
public void Debiter(double somme, Compte autreCompte) {  
    this.Debiter(somme);  
    autreCompte.Crediter(somme);  
}  
  
public void afficherResume() {  
    System.out.println("*****");  
    System.out.println("Numéro de Compte: " + this.code);  
    System.out.println("Solde de compte: " + this.solde);  
    System.out.println("Propriétaire du compte :");  
    // On utilise la méthode Afficher() du client  
    this.proprietaire.Afficher();  
    System.out.println("*****");  
}
```



10. Créer un programme de test pour la classe Compte.

Dans cet exercice nous allons créer la classe que nous nommons "TestBanque". Cette classe servira à tester notre programme.

Nous allons créer un objet **Client** avec les données saisies et l'objet **Compte** (**classecompte**), en lui donnant le "client1" que l'on vient de faire.

```
Client client1 = new Client(cin, nom, prenom, tel);

Compte compte1 = new Compte(client1);
```

Il faudra ensuite demander à l'utilisateur combien d'argent il veut déposer. Ensuite, nous allons appeler la méthode **Crediter()** du compte pour ajouter cet argent.

```
// Dépôt
System.out.print("\nSaisir le montant à déposer: ");
double montantDepot = input.nextDouble();

compte1.Crediter(montantDepot);
System.out.println("Opération bien effectuée");

// Affichage dépôt
compte1.afficherResume();
```

Faisons pareil pour le retrait en demandant le montant, puis en appelant la méthode **Debiter()** du compte.

```
// Retrait

System.out.print("\nSaisir le montant à retirer: ");
double montantRetrait = input.nextDouble();

compte1.Debiter(montantRetrait);
System.out.println("Opération bien effectuée");

// Affichage resume
compte1.afficherResume();
```

10. Résultat final

```
Compte 1:
Donner Le CIN: EE122112
Donner Le Nom: Dupont
Donner Le Prénom: hugues
Donner Le numéro de téléphone: 06.50.20.00.00
```

détails du Compte:

```
Numéro de Compte: 1
Solde de compte: 0.0
Propriétaire du compte :
CIN: EE122112
NOM: Dupont
Prénom: hugues
Tél: 06.50.20.00.00
*****
```

```
Saisir le montant à déposer: 3500
Opération bien effectuée
```

```
Numéro de Compte: 1
Solde de compte: 3500.0
Propriétaire du compte :
CIN: EE122112
NOM: Dupont
Prénom: hugues
Tél: 06.50.20.00.00
*****
```

```
Saisir le montant à retirer: 500
Opération bien effectuée
```

```
Numéro de Compte: 1
Solde de compte: 3000.0
Propriétaire du compte :
CIN: EE122112
NOM: Dupont
Prénom: hugues
Tél: 06.50.20.00.00
*****
```